ETX 方案 v3：用同样的 SOTA kernel，融合后更快

作者：Kailun Chen · 状态：v3，取代 `PLAN-moe.md` 和 `PLAN-fusion-win.md` · 日期：2026-09-26 · 依据：设计文档 15.9–15.10 节，`docs/RESEARCH-sync-hardware.md`，`docs/RESEARCH-moe-import.md`

0. 结论先行

- **要证明的一件事：** 把生产环境真正在用的 kernel（vLLM 的）原封不动放进 ETX 的一个持久化 megakernel，比同样这些 kernel 一个个 launch 更快。这样比较的只有"融合"本身，tile 写得好不好不参与比较。
- **已经做到的：** ETX 能原样导入 vLLM 的 kernel，GPU 上逐字节一致，完整模型 32/32 token 与 Hugging Face 一致。
- **还没做到的：** 融合版比同样的 kernel 逐个 launch 慢 7–16%。原因已经定位：ETX 的整卡同步比 kernel 边界贵。
- **修法：** 同步做三层处理。
- **少做：** 小 kernel 在每个 XCD 上各算一份，连续的小 kernel 合成一个任务。已写好。
- **做得便宜：** 跨 XCD 传递的小数据放进不经过 L2 的内存，每次同步不再需要写回和作废整个 L2。只动内存分配，不改 kernel。
- **等得快：** 每个 XCD 只留一个轮询者，调整休眠和优先级。
- **之后：** 在 MoE 模型上重复同样的比较，这是融合收益最大的场景；再扩展到小 batch 和多 GPU。
- **资源：** 单卡 MI300X 约 15 小时加 8 卡节点约 8 小时，前提是 GPU 随时可用，大约一周做完。当前余额 $1.78。

2026-09-27 上机后的更新（详见第 8 节）： dense 模型 batch 1 的融合作为负面结论收尾：vLLM 的 kernel 之间几乎没有空隙，而最便宜的正确全卡屏障（5.5 μs）仍是 kernel 边界（1.6 μs）的 3.4 倍。主线转到 MoE 的多请求：同一 CU 上的后台读取几乎不拖慢前台（1.00–1.04 倍），两个请求交错估计比 vLLM batch 2 的吞吐高约 30%。

1. 目标与验收标准

层级	标准
正确性（每一步都要满足）	每个导入的 launch 与 vLLM 原版逐字节一致（`--check`）；完整模型 32/32 token 与 HF greedy 一致
核心目标	同样的 kernel，ETX 融合版比 ETX 逐个 launch 快 **至少 10%**，并且快于 vLLM `custom_ops=all`（vLLM 用同一组 kernel）
延伸目标	快于 vLLM 默认配置（Inductor 生成的 norm/激活 kernel）；MoE 模型上快于 vLLM 最好成绩

2. 已有成果

导入工具 `etx/importer/`：

- 从 vLLM 源码里切掉 torch 的 host 代码，编成 AMDGPU IR。
- 把 `amdgpu_kernel` 改写成可内联的设备函数：参数从参数块读，block/线程 id 从 ETX 任务坐标换算，LDS 并进共用区。
- 一个 workgroup 里可以并排跑多个原版 block（slot），各有自己的屏障。
- 自动生成 ETX tile 和 host 端的参数布局，再和设备库打成一个 bundle。

已接入 12 个 vLLM kernel： `wvSplitK`、`paged_attention_rocm`（mfma4/mfma16 两种加 reduce）、`rms_norm`、`fused_add_rms_norm`、`rotary_embedding`、`reshape_and_cache`、`silu_and_mul`。模板实例和 launch 形状都按 vLLM host 端的选择逻辑推出。

实测（MI300X，batch 1，1024 上下文，ms/token）：

	Qwen2.5-1.5B	Qwen3-8B
vLLM 默认	1.862	4.906
vLLM `custom_ops=all`（同一组 kernel）	1.937	5.039
ETX 逐个 launch（同一份导入代码，HIP graph）	2.24–2.25	5.74
ETX 融合	2.61–2.62	6.16（2 个 token 的短测）*
ETX 融合，小 kernel 固定到单个 XCD	2.58	—

* 8B 融合版完整 32 token 那次运行失败，没有输出，原因未查清（见第 6 节）。其余各行都是 32/32 token 正确；8B 的短测 2/2 正确。

3. 诊断：两个差距

差距 A：融合比逐个 launch 慢 0.4 ms（8B）。

- 每个 token 要过 310–472 个整卡事件，每个比一次 kernel 边界多花约 1.2 μs。
- 根本原因在硬件：MI300X 的 8 个 XCD 各有一块 4 MB 的 L2，彼此不保持一致。ETX 的每个整卡事件都要生产者写回本 XCD 的 L2（`buffer_wbl2 sc1`），每个等待的 workgroup 再作废自己 L2 的非本地行（`buffer_inv sc1`）。kernel 边界每次 dispatch 只做一次，ETX 却是每个等待者都做一次。

差距 B：ETX 逐个 launch 比 vLLM 用同一组 kernel 慢 0.3–0.7 ms。 嫌疑按可能性排：

1. ETX 自写的 argmax 旧写法：每个线程约 150 次相互依赖的加载。已改成向量化。
2. 小 kernel 被塞进同一个 1024 线程的 workgroup：attention 的 40 个 block 只占 10 个 CU，vLLM 是 40 个。已加 slot 数配置。
3. 导入后编译器按常量 blockDim 重新优化，生成的代码和 vLLM 的二进制不同。`--check` 已经能逐 kernel 计时对比。

4. 技术方案

4.1 同步：少做、做得便宜、等得快

手段	内容	状态
按 XCD 复制（`ETX_VL_XCD`）	norm 和 SiLU 在 8 个 XCD 上各算一份（结果完全相同），写 XCD 本地的缓冲区；紧跟的 GEMV 排成 8×38 个任务，只等本 XCD 的事件。8B 的整卡事件从 472 个降到 254 个	已写好，本地编译零溢出，待测
合并小 kernel（`ETX_VL_CHAIN`）	q/k norm → RoPE → 写 cache 在一个任务里顺序执行，中间只隔 `__syncthreads`，每层少 3 个事件	已写好，待测

手段	内容	状态
uncached 通信内存	跨 XCD 传递的激活向量（每层几 KB）和事件计数器放进 uncached 内存（`hipDeviceMallocUncached`，MTYPE UC，不经过 L2）。release 只剩 `s_waitcnt vmcnt(0)`，acquire 只剩作废 L1（`buffer_inv sc0`）。权重运行中从不写，照常缓存。内存类型是页属性，**kernel 不改**	已写好（`ETX_VL_UC`，`ETX_COHERENT_DEVICE_DATA`；KV cache 的写入和读取固定在同一个 XCD），本地编译零溢出；是否运行由同步微基准判定。**预期已调低：** fleet 当年实测 uncached 计数器不比普通内存快（1.44 对 1.40 μs），绕过 L2 的激活在完整解码里没有收益，负载下一次跨 XCD 事件 6.16 μs 主要是显存争用（`RESEARCH-sync-hardware.md` 2.1）
每 XCD 一个轮询者（relay，P5）	304 个轮询者降到 8 个。之前测出分层 acquire 不安全，是因为 L2 里可能留着陈旧的行；数据放进 uncached 内存后就没有陈旧行了	代码已有，要和上一项一起测
等待方式	`s_sleep` 先短后长（目前固定 8，约 0.2 μs）；`s_setprio` 降低轮询 wave 的优先级；`s_wakeup` 提前叫醒同一 workgroup 里在睡的 wave	待调
硬件全局屏障 GWS	指令集里有：64 个资源，硬件排队，不用轮询。但 ROCm 在 gfx942 上自己也不用它，而且需要驱动给队列分配资源	只做一次微基准

4.2 追平 vLLM 的逐个 launch

用 `--check` 测出逐 kernel 的时间对比，确认差距 B 的构成，再用上面第 3 节的三项改动（argmax 向量化、slot 数配置、代码对比）逐项消除。

4.3 MoE（Qwen3-30B-A3B）

batch 1 时，每个 MoE 层 vLLM 只跑 6 个 kernel：

- router（`wvSplitK`）：已导入
- `topk_softmax`：HIP，可直接接
- 专家 gate_up：Triton `fused_moe_kernel`，192 个 program
- `silu_and_mul`：已导入
- 专家 down：Triton `fused_moe_kernel`，256 个 program
- `moe_sum`：HIP，可直接接

vLLM 在这个模型上只用到 26% 的带宽，是融合收益最大的地方。要补的有：

- **Triton 的 IR 用 LLVM 23 生成，hipcc 是 LLVM 22。** megakernel 的设备代码改用 LLVM 23 编（本地已在用这套），再用 `hipModuleLoad` 加载。
- **导入器支持 Triton 的写法：** 动态 LDS，以及末尾追加的 scratch 指针参数。
- **Triton kernel 可能超过 128 个 VGPR 的上限：** 先从缓存读出它的实际用量；超了就用 512 线程的 megakernel。
- **IR 要从 GPU 上 vLLM 实际跑过的 Triton 缓存里取：** 已加进上机脚本。

4.4 DeepSeek-V2-Lite：用 ETX 重排任务，填满闲置的带宽

依据（fleet 实测第 5 层，129.3 μs）： 前 5 个阶段（q/kv_a、attention、merge、o_proj、norm+router）共 73 μs，显存利用率 0–20%，时间花在每个任务 3–4 次约 5 μs 的读取往返上；专家阶段 56 μs，带宽 50–60%。整层的字节下限是 39 μs。

batch 1 能填进空闲窗口的只有共享专家（34.6 MB）。 路由专家（104 MB，占 62%）要等 router 算完才知道读哪几个。`examples/dsv2lite/whatif.py` 的估算：共享专家挪进 router 窗口、路由专家分到 8 个 XCD，最好情况每个 token 快 5%，前台被拖慢 1.3 倍或跨 XCD 事件要 6 µs 时就没有收益。

多请求才有大空间。 两个请求交错（A 的延迟阶段和 B 的专家阶段同时跑），前台不被拖慢时吞吐是 1.78 倍，单请求延迟多 12%；对照方案"两个 token 合读一次权重"是 1.57 倍、多 28%。前台被拖慢超过 1.14 倍，交错就不如合读。两者都要和 vLLM 的 batch 2 比。

本地已查明的限制： 交错要求每个 CU 同时驻留两个 workgroup，也就是每条 lane 不超过 256 个寄存器。现在 megakernel 是 286（fleet 的 GEMV 路径 262，来自双缓冲的行数×深度，和 `FLEET_STREAM_DEPTH` 无关），强制压到 256 会溢出 41 个。

下次上机要测的（都已写好）：

测量	代码	决定什么
同一个 CU 上有流式读取时，64 KB 读取往返慢多少（深度 0/1/2/4/8），以及分开 CU 时	`bench/overlap/overlap_bench.hip fold`	f：共享专家挪窗口、交错是否划算
专家大小的数据从 Infinity Cache 读和从显存读的速度	`overlap_bench stream`	预取有没有意义
ETX 版 DeepSeek 的逐任务 trace	`ETX_TRACE_RAW` + `python -m etx.tools.timeline`	每个 XCD 的忙闲、关键路径、闲置带宽的窗口
vLLM 在 DeepSeek 和 Qwen3-30B-A3B 上 batch 1/2/4	`examples/llm/vllm_bench.py --batch`	多请求方案的对照

事先定好的门槛： `whatif.py` 读入实测后估算每 token 快 5% 以上，才做 batch 1 的重排；交错要满足 f 低于它打印的合读临界值，并且吞吐高于 vLLM batch 2，才改寄存器和 host。

4.5 之后

- **小 batch（4–16）和更长上下文：** 这是 serving 的常见场景。
- **多 GPU：** 把通信和计算融合在一起，对 kernel 边界来说这是最难的地方，megakernel 的优势最大。

5. 执行计划与 GPU 需求

阶段	天数	内容	完成标准	GPU
1	第 1–2 天	`vm_next.sh`：先跑同步微基准（`bench/sync/sync_bench.hip`）和 overlap 微基准（`bench/overlap/overlap_bench.hip`），按事先定好的门槛决定是否测 uncached 变体；再抓 vLLM trace 定天花板；然后逐 kernel 计时和各变体整模型；DeepSeek 的 trace、调度分析和 vLLM batch 1/2/4	8B 和 1.5B 上，融合版比逐个 launch 快 10% 以上，32/32 正确；DeepSeek 各方案的估算有实测输入	单卡约 3 小时
2	第 2–4 天	MoE：接入 `topk_softmax` 和 `moe_sum`；LLVM 23 设备编译 + `hipModuleLoad`；导入 Triton `fused_moe_kernel`，`--check` 验证，然后跑整个模型	Qwen3-30B-A3B 融合版快于 vLLM	单卡约 6 小时
3	第 4–5 天	batch 4–16，更长上下文	每 token 时间优于 vLLM 同 batch	单卡约 6 小时

阶段	天数	内容	完成标准	GPU
4	第 5–7 天	多 GPU：张量并行/专家并行，通信融合	8 卡上优于 vLLM	8 卡约 8 小时

合计：单卡约 15 小时加 8 卡约 8 小时。本地能做的开发都放在上机之前完成，GPU 时间只用来验证和测量。

最理想的资源形式： 一台能随开随关的单卡 MI300X，带持久磁盘，模型和 vLLM 镜像不用每次重新下载，省掉每次开机约 15 分钟的准备时间；第 5–7 天再加一块 8 卡节点的时间。

6. 风险与应对

风险	应对
uncached 内存带来的延迟抵消了同步节省	只放每层几 KB 的向量和计数器，先用微基准量化；不行就退回"按 XCD 复制 + 合并"，只保留 XCD 本地的同步
同步修好后，融合仍然没有赢	那就说明 batch 1 dense 解码本身就是一条屏障链，没有重叠空间（之前手写 tile 时就这么判断过），重点转向 MoE 和多 GPU，那里有 kernel 边界做不到的重叠
Triton kernel 在 megakernel 里溢出寄存器	512 线程 megakernel；或者只把 MoE 层单独做成一个 megakernel
8B 融合版完整运行失败（未查清）	下次上机保留完整日志，不再 grep 过滤；看门狗超时时打印事件状态
GPU 拿不到	本地先把代码准备好；每次上机按脚本一次跑完；最好有专用机器

7. 已经准备好的东西

- 代码：`github.com/metalopscloud/amdproject`（`main` 分支），也同步在 `cklxx/etx-megakernel`。
- 上机脚本：`examples/vllm_llm/vm_next.sh`。一次跑完两个模型、6 个同步变体、逐 kernel 计时对比、vLLM 的 kernel trace、DeepSeek 的 trace 与调度分析、vLLM 多 batch 成绩，最后采集 MoE 的 Triton 缓存。完整日志保留在 `~/vlnext/`。本地预检：`examples/vllm_llm/local_matrix.sh`。
- 调度检测：`python -m etx.tools.timeline`（每个 XCD 的忙闲、关键路径、显存需求曲线、HTML 时间线；也能分析模拟出的排布）。
- 调研：`docs/RESEARCH-sync-hardware.md`（同步的硬件手段）、`docs/RESEARCH-moe-import.md`（MoE 导入路线）。
- 设计文档：`docs/ETX_Technical_Design.md` 15.8–15.10 节（导入方法与实测）。

8. 2026-09-27 上机结果与下一步

双卡 MI300X VM，约 50 分钟，花费 $5.98（1 小时最低收费），余额 $45.75。所有模型运行 32/32 token 正确；DeepSeek 另有 27/27 层通过。数据在 `examples/vllm_llm/results/2026-09-27/`，设计文档 15.11 节。

问题	实测	结论
ETX 同步能否比 kernel 边界便宜	kernel 边界 1.6 µs；全卡屏障 7.8 µs；uncached + 正确 acquire 5.5 µs	不能。dense batch 1 融合收尾
vLLM kernel 之间有多少空隙	8B：kernel 忙碌 5.08 ms，实测 5.04 ms/token	几乎为零，只剩小 kernel 自身约 4 µs 的固定开销
融合各变体（ms/token）	1.5B 最好 2.248（逐个 launch 2.073，vLLM 1.937）；8B 最好 5.864（5.677，5.039）	仍慢 3–8%，比 vLLM 慢 16%
同 CU 后台读取拖慢前台多少	1.00–1.04 倍	交错划算
Infinity Cache 比显存快吗	不快	不做预取
DeepSeek：fleet / ETX / vLLM	3.56 / 3.74–3.79 / 4.195 ms；vLLM batch 2、4 为 376、688 tok/s	ETX 慢 fleet 5%，原因未完全定位

发现并修复的正确性问题： `buffer_inv sc0` 不作废 L1。ETX 的 XCD 内事件和 fleet 都用了它，靠 L1 被权重冲掉才没出错。ETX 默认改为 `buffer_inv sc1`（8B 无代价，DeepSeek 慢 7%），旧写法保留为 `-DETX_ACQUIRE_DOMAIN_RELAXED`。

下一步（按事先定好的门槛）： 1. DeepSeek 两个请求交错：前提是 megakernel 每条 lane 不超过 256 个寄存器（现在 286，来自 fleet GEMV 的双缓冲）。先在本地把 GEMV 调到 256 以内并确认零溢出，再做 host 端两路请求。估计吞吐约 490 tok/s，对照 vLLM batch 2 的 376。 2. 同时评估"两个 token 合读一次权重"（估计约 440 tok/s，不需要改寄存器），两者取其一。 3. 找出 ETX 比 fleet 慢的 0.18 ms：用两份全 token trace 逐层对比。 4. MoE 导入：Triton `fused_moe_kernel` 的 IR 已采集（`results_2026-09-27/vlnext/moe/triton`）。

第二次上机（同日，约 14 分钟，$5.98，余额 $39.77，设计文档 15.12）： 按门槛决定做 DeepSeek 两个请求交错。寄存器限制那个构建事后查明并未真正压到 256（256 VGPR + 14 AGPR，仍是每 SIMD 一个 wave），0.8% 不作数；原因是 fleet 的 GEMV 在运行时分派 5 条路径，合并后要 262 个寄存器，打补丁把路径拆成独立函数后能压到 256（溢出 100 个），代价下次上机实测；两个不同请求实际要读 11.38 个路由专家（均匀路由估计 11.44），合读省不了字节，交错估计 1.72–1.78 倍对合读 1.57 倍。ETX 比 fleet 慢 0.18 ms/token 的原因：XCD 内三次交接每层共约 7 µs（fleet 让最快的 XCD 先走），另有 kv_a 分工方式不同（影响相互抵消）。

第三次上机（同日，约 10 分钟，1 小时最低收费 $5.98，余额 $33.79，设计文档 15.13）： 两个请求在同一个 megakernel 里交错运行，结果全部正确（两个请求都 32/32、27/27）。最好的是不加耦合：366 tok/s，比 fleet 单请求高 30%，比 ETX 单请求高 37%，但比 vLLM batch 2 的 376 tok/s 低 3%，没有过门槛。估算（1.72–1.78 倍）没有兑现：两个请求会自动跑成同步，同一阶段一起跑、互相拖慢；强制错开半层的两种耦合更慢（314、341 tok/s）。下一步要赢 vLLM 的 batch，需要让两个 token 共用延迟阶段的工作（同一次 q/kv_a、o_proj、attention 准备），而不是指望它们重叠。

第四、五次上机（同日，约 16 分钟，$5.98，余额 $27.81）：自下而上建模。 逐任务的操作清单（`docs/OPS-dsv2.md`）、硬件原语微基准和真实 kernel 的逐阶段隔离测量，把每层时间逐项对上（`docs/MODEL-dsv2.md` 第 5 节）。找到并修复了最大的一项：ETX 的事件计数器 8 个挤在同一个缓存行，每次 XCD 内交接多花约 2.3 µs。每个计数器独占一行后（`ETX_EV_STRIDE` 32，gfx942 默认）：DeepSeek 单请求 3.72 → 3.47 ms/token，**比 fleet 的 3.55 快 2%**；两个请求 365 → 416 tok/s，**比 vLLM batch 2 的 376 高 10.6%**，每个请求的延迟 4.76 ms 也低于 vLLM 的 5.32。全部 32/32 正确。